

Valid Parenthesis String Logic

Absolutely. The key to understanding this greedy solution is to first understand **what the recursion/DP would keep track of**, and then see how this code compresses that information into just **two numbers**.

This is the classic **Valid Parenthesis String** problem where:

- (→ must be open
-) → must be close
- * → can be (,), or empty

1. First think about recursion

Suppose we have:

```
s = "(*)"
```

When recursion sees *, it has **3 choices**:

```
* → '('
* → ')'
* → ''
```

So conceptually:

```
C++
bool solve(i, open)
```

means:

"I am at index **i**, and currently I have **open** unmatched (brackets."

For every character:

If (

```
open → open + 1
```

If)

```
open → open - 1
```

If *

We branch:

```

      /      *      \
    '('      '|'      ')'
    |         |         |
open + 1    open - 1    open

```

So recursion explores **many possible values of open**.

2. The problem with recursion

Consider:

```
"(*(*)*)"
```

At every *, there are 3 possibilities.

If there are **k** stars, recursion can have roughly:

```
3^k
```

possibilities.

But notice something important:

We don't actually care about **every individual possibility**.

We only care about:

What is the **minimum possible number** of unmatched (?

and

What is the **maximum possible number** of unmatched (?

That's where this greedy solution comes from.

3. Instead of storing every open, store a range

Suppose after processing some prefix, recursion can produce:

```
open = 0
open = 1
open = 2
open = 3
```

Instead of storing:

```
{0, 1, 2, 3}
```

we store:

```
minOpen = 0
maxOpen = 3
```

So:

```
[minOpen, maxOpen]
  ↓
  [0, 3]
```

means:

Depending on how we interpreted the *, the number of currently unmatched (can be anywhere from 0 to 3.

This is the **main intuition** behind the entire code.

4. What happens with (?

Suppose currently:

```
[minOpen, maxOpen] = [1, 3]
```

Now we see:

```
(
```

An opening bracket **always increases** unmatched opens by 1.

Therefore:

```
[1, 3]
```

becomes:

```
[2, 4]
```

That's why:

```
C++
if (c == '(') {
    minOpen++;
    maxOpen++;
}
```

5. What happens with) ?

A closing bracket **always decreases** unmatched opens by 1.

So:

```
[2, 4]
```

becomes:

```
[1, 3]
```

Hence:

```
C++
else if (c == ')') {
    minOpen--;
    maxOpen--;
}
```

6. The interesting part: *

This is where the recursion gets compressed.

Suppose:

```
[minOpen, maxOpen] = [2, 5]
```

We encounter:

```
*
```

Recursion gives us 3 possibilities.

Case 1: * = '('

```
open + 1
```

So range becomes:

```
[3, 6]
```

Case 2: * = ')''

```
open - 1
```

So range becomes:

```
[1, 4]
```

Case 3: * = ''

```
open
```

So:

```
[2, 5]
```

Now combine all possible results:

```
[1, 6]
```

Therefore:

```
C++
minOpen--;
maxOpen++;
```

That's exactly what the code does.

7. Why does * only do this?

This:

```
C++
minOpen--;
maxOpen++;
```

looks strange at first.

But think:

```

minimum possible open
  ↓
choose '*'
  ↓
make it ')'
  ↓
-1

```

Therefore:

```

C++
minOpen--;

```

And:

```

maximum possible open
  ↓
choose '*'
  ↓
make it '('
  ↓
+1

```

Therefore:

```

C++
maxOpen++;

```

We don't explicitly consider the empty case because it lies **between** those two possibilities.

For example:

```

open = 5
* = ')' → 4
* = '(' → 5
* = '(' → 6
range = [4,6]

```

So we only need the boundaries.

8. Now the most important line

```

C++
if (maxOpen < 0)
    return false;

```

Why?

Suppose:

```

s = ")"

```

Initially:

```

minOpen = 0
maxOpen = 0

```

See):

```

minOpen = -1
maxOpen = -1

```

Now:

```

maxOpen < 0

```

means:

Even under the most optimistic interpretation, we have more) than (.

There is **no possible interpretation** that can fix this.

So immediately:

```

C++
return false;

```

9. Why can `minOpen` become negative?

This is slightly different.

Suppose:

```
s = "("
```

After (:

```
[1,1]
```

Then *:

```
minOpen = 0
maxOpen = 2
```

Fine.

But consider:

```
s = "*)"
```

Start:

```
[0,0]
```

*:

```
[-1,1]
```

But -1 doesn't actually mean:

We have -1 opening brackets.

That's impossible.

It means:

The minimum branch tried to use * as), but there weren't enough (available.

So we clamp:

```
C++
minOpen = max(minOpen, 0);
```

giving:

```
[0,1]
```

This means:

The minimum possible unmatched opening brackets is 0.

10. Why don't we clamp `maxOpen`?

Because `maxOpen` represents:

The maximum number of unmatched (that can possibly exist.

If it becomes negative:

```
maxOpen < 0
```

then even the maximum possibility is invalid.

So we return false.

But if:

```
minOpen < 0
```

that's okay because some interpretations may have fewer than zero conceptually, but those interpretations are simply impossible. We discard that invalid lower portion by doing:

```
C++
minOpen = max(minOpen, 0);
```

11. Finally: return minOpen == 0

At the end, we have:

```
[minOpen, maxOpen]
```

Suppose:

```
[0, 3]
```

Then there exists **at least one interpretation** where:

```
open = 0
```

Therefore the string can be valid.

So:

```
C++
return minOpen == 0;
```

If we have:

```
[2, 5]
```

then even the minimum possible unmatched opens is 2.

Therefore:

```
No interpretation can close everything.
```

Return:

```
false
```

12. Now connect it directly to recursion

This is the most important connection.

Recursive approach

You would maintain:

```
C++
open
```

and explore:

```
*
├ '(' → open + 1
├ ')' → open - 1
└ ',' → open
```

So recursion creates many possible **open** values.

For example:

```
Possible opens:
0
1
2
3
```

Instead of storing all of them, greedy says:

```
minimum = 0
maximum = 3
```

So:

```
Recursion:
{0,1,2,3}
```

Greedy:
[0,3]

Then as we process characters, we update the boundaries.

13. Think of `minOpen` and `maxOpen` as two extreme recursion paths

This is probably the easiest way to remember the code.

`minOpen`

Always ask:

"What is the smallest number of unmatched (I can have?"

For *, choose:

```
*
↓
','
```

because that decreases opens.

Therefore:

```
C++
minOpen--;
```

`maxOpen`

Always ask:

"What is the largest number of unmatched (I can have?"

For *, choose:

```
*
↓
'('
```

because that increases opens.

Therefore:

```
C++
maxOpen++;
```

So:

```
minOpen = most ')' interpretation
maxOpen = most '(' interpretation
```

14. Dry run

Let's take:

```
s = "(*)"
```

Start:

```
min = 0
max = 0
```

Character 1: (

```
min = 1
max = 1
```

Range:

```
[1,1]
```

Character 2: *

Minimum → treat as):

```
min = 0
```

Maximum → treat as (:

```
max = 2
```

Range:

```
[0,2]
```

Character 3:)

```
min = -1
max = 1
```

Clamp:

```
min = 0
```

Range:

```
[0,1]
```

Character 4:)

```
min = -1
max = 0
```

Clamp:

```
min = 0
```

Final:

```
[0,0]
```

Therefore:

```
C++
minOpen == 0
```

→ true.

And indeed:

```
(*)
↓
()
```

is valid.

15. The complete mental model

Don't memorize the code. Memorize this:

Recursion asks:

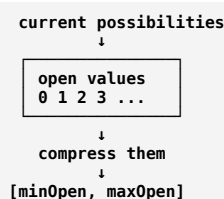
"What can open be?"

Greedy asks:

"What is the minimum possible open?"

"What is the maximum possible open?"

Therefore:



And every character modifies this range:

```
'(' → [min+1, max+1]
')' → [min-1, max-1]
'*' → [min-1, max+1]
```

Then:

```
maxOpen < 0
↓
impossible immediately
minOpen = max(minOpen, 0)
↓
remove impossible negative states
end:
minOpen == 0
↓
there exists a valid interpretation
```

So the greedy solution is basically an optimized recursion/DP where instead of tracking every possible open state, we track only the smallest and largest possible open.

Sure. Let's dry run (****) exactly using minOpen and maxOpen.

String:

```
( * ) ) )
0 1 2 3 4
```

Initial:

```
minOpen = 0
maxOpen = 0
```

Step 1 — (

(must be an opening bracket:

```
minOpen++
maxOpen++
```

So:

```
minOpen = 1
maxOpen = 1
```

Range:

```
[1, 1]
```

Meaning: we definitely have **1 unmatched** (.

Step 2 — *

* has 3 possibilities:

```
* → '('
* → ')'
* → ''
```

For the **minimum**, choose):

```
minOpen = 1 - 1 = 0
```

For the **maximum**, choose (:

```
maxOpen = 1 + 1 = 2
```

So:

```
[0, 2]
```

Meaning:

```
possible unmatched '(' = 0, 1, or 2
```

Step 3 —)

A) decreases the number of open brackets.

```
minOpen = 0 - 1 = -1
maxOpen = 2 - 1 = 1
```

So temporarily:

```
[-1, 1]
```

But negative open brackets are impossible, so:

```
C++
minOpen = max(minOpen, 0);
```

Therefore:

```
[0, 1]
```

Step 4 —)

Again:

```
minOpen = 0 - 1 = -1
maxOpen = 1 - 1 = 0
```

So:

```
[-1, 0]
```

Clamp minOpen:

```
[0, 0]
```

Still potentially valid.

Step 5 — final)

Again:

```
minOpen = 0 - 1 = -1
maxOpen = 0 - 1 = -1
```

Now:

```
maxOpen < 0
```

So immediately:

```
C++
return false;
```

Final table

Character	minOpen	maxOpen	Range
Start	0	0	[0,0]
(	1	1	[1,1]
*	0	2	[0,2]
)	0	1	[0,1]
)	0	0	[0,0]
)	-1	-1	✗

At the final):

```
maxOpen = -1
```

means **even the most favorable interpretation of * cannot save us.**

So:

```
(*) )
↓
```

INVALID

The recursion connection

For the *, the best possible choice is:

```
(*)
  ↓
(() )
```

After choosing * = '(', we get:

```
( )
```

Then:

```
( ( ) )
```

There are **3 closing brackets after the star**, but only **2 opening brackets total** ((+ *).

So one) will always remain unmatched.

That's exactly why `maxOpen` eventually becomes -1.

Yes — the **core intuition** is actually very simple:

We don't try to decide what * is immediately. We keep all possibilities alive using a range.

For (*)), let's forget the code for a moment.

1. What does recursion do?

When we see *, recursion says:

```
* can be:
  '('
  ''
  ')'
```

So there are multiple possible numbers of open brackets.

Instead of exploring all of them, we ask:

```
What is the MINIMUM number of open brackets possible?
What is the MAXIMUM number of open brackets possible?
```

That's:

```
[minOpen, maxOpen]
```

2. Why does this work?

Suppose we have:

```
(*
```

The * can give us:

```
* = ')' → open = 0 ← minimum
* = ''  → open = 1
* = '(' → open = 2 ← maximum
```

So instead of storing:

```
0, 1, 2
```

we store:

```
[0, 2]
```

That's the entire trick.

3. Then what are we checking?

As we process the string, we're asking:

"Is there still at least ONE possible interpretation that can become a valid parenthesis string?"

For:

```
(*) ) )
```

after processing (*), we have:

```
[0, 0]
```

Meaning:

There is still one possible interpretation where everything is balanced.

But then the final) comes:

```
[0, 0]
  ↓
[-1, -1]
```

Now even the **maximum possible** number of open brackets is -1.

That means:

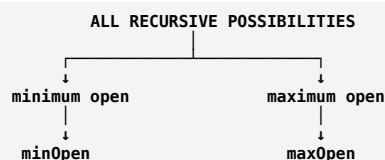
There is no possible interpretation left.

Therefore:

```
false
```

The easiest way to remember it

Think of `minOpen` and `maxOpen` as two "boundaries" of recursion:



We don't care about every recursive branch.

We only care about:

```

    "How low can open go?"
      ↓
    [ MIN , MAX ]
      ↓
    "How high can open go?"
  
```

And * is basically our **flexibility**:

```

* as ')' → decrease MIN
* as '(' → increase MAX
  
```

So the most important intuition is:

Greedy doesn't choose what * is. It postpones the choice and maintains the entire possible range of choices.

That's why this solution is much more powerful than simply saying "* should be (" or "* should be)."

1. Core Intuition

Instead of exploring all possibilities recursively, we only keep a RANGE of possible number of open brackets.

Recursion would explore many "open" values

```

s = ( { * } ) }
      |   |   |
      v   v   v
    open = 2  open = 0  open = 1
  
```

Many more states as we go further...

Hard to track all!

Greedy Idea: Track Only Range

At any point, the number of open brackets can be anywhere between:

$$\text{minOpen} \leq \text{open} \leq \text{maxOpen}$$

We only store the boundaries.

Example at index 1 (after "("):

Possible open values:

```

0 1 2 3 4 ...
| | | | |
minOpen = 0 maxOpen = 2
  
```

So we store range [0, 2]

Why does this work?

If at the end, $\text{minOpen} = 0$, it means there exists at least one way to make $\text{open} = 0$ (all brackets closed).

If even $\text{maxOpen} < 0$ at some point, it means even in the best case we have more ')' than '('.

$\text{minOpen} = 0 \Rightarrow \text{Valid}$

$\text{maxOpen} < 0 \Rightarrow \text{Invalid (return false)}$

2. How do we update the range?

When we see '('

It must increase open count.

```

0 1 2 3 ...
| | | |
New Range: [min+1, max+1]
  
```

But minOpen can't be negative. So we do: $\text{minOpen} = \max(\text{minOpen}, 0)$

When we see ')' or '*'

It must decrease open count.

```

0 1 2 3 ...
| | | |
New Range: [min-1, max-1]
  
```

As ')' $\rightarrow$ decrease As '(' $\rightarrow$ increase As '*' $\rightarrow$ no change

If $\text{maxOpen} < 0$ at any time $\rightarrow$ return false (too many ')')

3. Dry Run for s = ("*)

Index	Char	What happens?	minOpen	maxOpen	Range [minOpen, maxOpen]	Explanation
Start	-	Initial values	0	0	[0, 0]	No characters processed yet
0	(	Increase both	1	1	[1, 1]	One '(' $\rightarrow$ open = 1
1	*	min--, max++	0	2	[0, 2]	* as ')' $\rightarrow$ open = 0 (min) * as '(' $\rightarrow$ open = 2 (max) * as '*' $\rightarrow$ open = 1 (in between)
2	)	Decrease both	0	1	[0, 1]	After ')', open decreases by 1
3	)	Decrease both	0	0	[0, 0]	Still possible to be valid
4	)	Decrease both	-1	-1	[-1, -1]	$\text{maxOpen} < 0 \rightarrow$ Invalid! Even best case has more ')' than '('

4. Key Intuition (Big Picture)

Recursion asks: "What can open be?" (many values)

Greedy asks: "What is the minimum possible open?"

What is the maximum possible open?"

If range ever becomes invalid ($\text{maxOpen} < 0$) $\rightarrow$ impossible
If range includes 0 as the end ($\text{minOpen} = 0$) $\rightarrow$ possible!

ChatGPT can make mistakes. Check important info.